

Set (Insiemi)

Set (Insiemi)

Raccolte di elementi unici — perfetti per eliminare i duplicati e fare operazioni sugli insiemi.

Cosa sono i set?

Un **set** (insieme) è una raccolta di elementi con due regole fondamentali:

1. **Nessun duplicato**: ogni elemento può comparire al massimo una volta
2. **Nessun ordine garantito**: gli elementi non hanno una posizione fissa

Pensa a un set come a un sacchetto di palline colorate: sai quali colori ci sono dentro, ma non puoi dire "dammi la terza pallina".

Creare un set

Si usano le **parentesi graffe** `{}`, come per i dizionari, ma senza le chiavi:

```
frutta = {"mela", "banana", "ciliegia"}
numeri = {1, 2, 3, 4, 5}
```

⚠ `{}` da solo crea un **dizionario vuoto**, non un set vuoto! Per creare un set vuoto usa `set()`:

```
vuoto = set()      # set vuoto ✓
anche_vuoto = {}   # dizionario vuoto, non un set! ✗
```

...

I duplicati vengono rimossi automaticamente:

```
con_duplicati = {1, 2, 2, 3, 3, 3}
print(con_duplicati)  # {1, 2, 3} - solo valori unici
```

Aggiungere e rimuovere elementi

```
frutta = {"mela", "banana"}

frutta.add("ciliegia")      # aggiunge un elemento
frutta.remove("banana")    # rimuove (dà errore se non esiste)
frutta.discard("uva")      # rimuove senza errore se non esiste
```

Controllare se un elemento è presente

```
frutta = {"mela", "banana", "ciliegia"}
print("mela" in frutta)    # True
print("uva" in frutta)     # False
```

I set sono molto veloci per questo tipo di ricerca, anche più delle liste.

Operazioni sugli insiemi

Qui i set diventano particolarmente utili: puoi fare operazioni tipiche della matematica degli insiemi.

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

Unione — tutti gli elementi di A e di B:

```
print(A | B)    # {1, 2, 3, 4, 5, 6}
```

Intersezione — solo gli elementi presenti in entrambi:

```
print(A & B)    # {3, 4}
```

Differenza — gli elementi in A ma non in B:

```
print(A - B)    # {1, 2}
```

Il trucco più utile: eliminare i duplicati

Il modo più semplice per eliminare i duplicati da una lista è convertirla in set e poi riconvertirla in lista:

```
lista = [1, 2, 2, 3, 3, 4, 1]
lista_pulita = list(set(lista))
print(lista_pulita)  # [1, 2, 3, 4]
```

:::note L'ordine originale non viene preservato, perché i set non hanno un ordine fisso. :::

Limiti dei set

- Non puoi accedere agli elementi per posizione: `frutta[0]` causa errore
- Gli elementi devono essere immutabili: puoi mettere numeri, stringhe, tuple, ma non liste